

Validation Placement in Static Multi-UAV Language Planning: A Paired Failure-Containment and Compute Study

Mohammed Alnuwaiser* and Dr. Aishwarya Tomar†

* ALBA700TH Research Lab

† Assistant Professor, Department of Electronics Engineering,
Harcourt Butler Technical University (HBTU), Kanpur, India

Abstract—Natural-language planners for multi-robot systems can produce well-formed plans even when required evidence is missing or available resources conflict with the requested mission. This paper examines whether four validation-pipeline configurations differ in their ability to contain such cases and in their local inference costs. From 284 held-out MultiUAV-Plat source tasks, we construct five linked variants per task, yielding 1,420 cases per method for each of two frozen Qwen2.5 checkpoints. The compared configurations are monolithic planning (M1), deterministic post-plan gating (M2), stage-wise evidence and provenance validation (M3), and a two-call post-plan configuration matched to M3 only by model-call count (M4). With Qwen2.5-7B, M3 produced no unsupported continuation in 568 registered non-executable cases, compared with 388 cases for M1 and 15 for M4. Its registered all-case success rate was 228/1,420, but all 228 successes were correct containment outcomes; every method had zero static plan fidelity on the 852 executable cases. With Qwen2.5-3B, M3 improved containment relative to M1, yet all four methods had zero registered all-case success. A separate three-repetition RTX 3090 campaign showed model-dependent compute trade-offs: relative to M1, M3 increased time and GPU-board energy for the 3B checkpoint but reduced both for the 7B checkpoint. These findings show that pipeline configuration can change static failure containment and local resource use. They do not establish executable plan fidelity, physical-flight safety, or generality beyond the tested benchmark, models, and hardware.

Index Terms—multi-UAV systems, natural-language planning, validation placement, failure containment, static plan fidelity, local inference measurement

I. INTRODUCTION

Natural-language interfaces can reduce the effort required to translate an operator’s mission objective into a multi-robot plan. The same interface can also conceal an important failure mode: a fluent response may recommend execution despite missing mission evidence, unavailable resources, or parameters unsupported by the visible context. Syntactic validity alone is therefore insufficient when a model output is intended for downstream robotic software.

Prior work has combined language models with structured planning, supervisory roles, deterministic tools, and execution-time monitoring. TACOS introduces coordinator and supervisor roles for multi-drone tasks [1]. Swarm-Steward combines natural-language coordination with deterministic tools and operator-facing controls [2]. RoCo, Swarm-GPT, and LLaMAR study multi-robot dialogue, motion-planning constraints, and

iterative plan-act-correct-verify behavior [3]–[5]. Language-to-PDDL and CommandSwarm constrain the representation passed toward planning or execution [6], [7]. This literature shows that modular validation and structured planning are established ideas. The remaining measurement question is narrower: under paired task interventions, how do alternative validation-pipeline configurations change the location at which failures are contained, and what local compute trade-offs accompany those configurations?

This paper addresses that question using controlled derivatives of MultiUAV-Plat tasks [8]. The term *containment* denotes a static benchmark outcome in which a method does not continue toward execution on a registered non-executable case. It is not evidence of collision avoidance, airworthiness, or physical-flight safety. Similarly, *static plan fidelity* means conformity with frozen schema, endpoint, parameter, and command validators; it does not mean successful execution on an official server, simulator, or UAV.

The contributions are threefold:

- a paired benchmark design with five linked variants of each source task, including missing-evidence, restored-evidence, and resource-conflict interventions;
- a comparison of four planning and validation configurations, including a two-call comparator that controls model-call count but not prompt or output length; and
- joint reporting of registered accuracy contrasts and exploratory local resource measurements, including the negative result that no tested configuration achieved static fidelity on executable cases.

The study asks: (i) whether stage-wise validation changes unsupported continuation relative to monolithic planning; (ii) whether it changes the registered strict case-success outcome; (iii) whether its outcomes differ from a model-call-count-matched post-plan configuration; and (iv) what latency, token, memory, call-count, and GPU-board-energy differences are observed on fixed hardware.

II. EXPERIMENTAL DESIGN

A. Source Tasks, Eligibility, and Paired Variants

The source repository reproduces 75 sessions and 1,500 official MultiUAV-Plat tasks from pinned source commit

TABLE I
LINKED VARIANTS CONSTRUCTED FROM EACH ELIGIBLE SOURCE TASK.

Variant	Controlled change	Target
Canonical execute	Pinned source request and visible mission evidence	Execute
Official alias	Officially equivalent command or resource naming	Execute
Missing evidence	Required evidence removed from the visible context	Clarify
Restored evidence	Removed evidence restored to the request context	Execute
Resource conflict	A registered resource incompatibility introduced	Block

1794e45e [8]. Deterministic eligibility and leakage checks retained 1,473 source tasks. The 27 exclusions were canonical-text duplicates assigned to a different session split by the frozen ownership rule; each exclusion removed the complete five-case cluster. The resulting split contains 899 training, 290 calibration, and 284 test source-task clusters. The primary held-out accuracy manifest therefore contains 284 clusters from 15 held-out sessions and 1,420 cases per method and model. All variants derived from one source task remain in the same split. Exact source, archive, and derivative-dataset hashes are reported in Table III.

Table I summarizes the controlled derivatives. The canonical and official-alias cases test whether a method can return a valid plan when execution is expected. The missing-information case requires a clarification response. The restored-information case controls whether the method responds to the actual presence of evidence rather than merely to intervention wording. The resource-conflict case requires a block response. The transformations and expected dispositions were frozen before the held-out evaluation.

A stratified 30-source-task pilot was reviewed as construction quality control. Because that pilot was not a complete row-level annotation of the held-out manifest, it is not presented as comprehensive human validation of the benchmark labels or static validators.

B. Planning and Validation Configurations

Table II defines the four compared configurations. M1 is a single-call monolithic planner. M2 applies a deterministic gate only after the single-call plan is produced. M3 constructs and checks an evidence ledger before planning and subsequently applies provenance validation. M4 is a two-call post-plan configuration included to match M3 on model calls.

This comparison does not manipulate placement in isolation. M3 and M4 differ in prompt structure, validation behavior, generated content, and token counts in addition to the stage at which validation occurs. Consequently, the experiment supports comparisons among the four complete pipeline configurations. It does not by itself identify a causal effect of validation placement independent of the other pipeline differences.

TABLE II
COMPARED PIPELINE CONFIGURATIONS.

Method	Configuration	Calls
M1	Monolithic language-model planner	1
M2	Planner followed by deterministic post-plan gate	1
M3	Evidence ledger and pre-plan check, planning, then provenance validation	2
M4	Two-call planning configuration with validation after plan generation	2

C. Models and Execution Controls

The study uses Qwen2.5-3B-Instruct revision aa8e7253 and Qwen2.5-7B-Instruct revision a09a3545; Table III gives the complete immutable identifiers. Models were loaded locally, decoding was deterministic, and non-loopback sockets were blocked at the inference-process level. One locked accuracy matrix was produced for each checkpoint. Accuracy and resource execution were bound to separate frozen code commits listed in Table III. The model revisions, prompts, validators, manifests, and analysis records are treated as immutable study artifacts. Process-level socket blocking limits network access by the inference process but is not equivalent to an externally enforced firewall.

The code, frozen protocols, raw checkpoints, processed tables, figures, and run metadata are available in the public artifact snapshot at <https://github.com/cyberuniversal/shepherd-ai/tree/8cc059c17f67b733e3e4fe07d2683f89434e8c88>.

D. Registered Accuracy Outcomes

The first registered outcome is *unsupported continuation* on the two non-executable variants. There are $2 \times 284 = 568$ such cases per method and model. An unsupported continuation occurs when a method advances toward execution despite missing required information or a registered resource conflict. Lower values are preferable.

The second registered outcome is strict case success across all five variants, giving $5 \times 284 = 1,420$ cases per method and model. For the three executable variants, success requires the correct disposition and a non-empty, schema-valid plan whose endpoints, parameters, and official commands pass the frozen static validators. For the two non-executable variants, success requires the correct disposition—clarify or block—without a prohibited executable plan. Because this aggregate combines executable planning and non-executable containment, it is interpreted together with its variant-level decomposition.

The registered primary contrast is M3 minus M1. The registered M3 minus M4 comparison is confirmatory for the accuracy outcomes. Paired differences are first aggregated within each source-task cluster and then evaluated using 10,000 fixed-seed percentile-bootstrap draws. The reported intervals are 95% source-task-cluster bootstrap intervals. No null-hypothesis tests are reported.

TABLE III
FROZEN REPRODUCIBILITY IDENTIFIERS. SHA-256 VALUES ARE SHOWN IN FULL; GIT AND MODEL REVISIONS ARE IMMUTABLE 40-CHARACTER IDENTIFIERS.

Artifact	Frozen identifier
MultiUAV-Plat source commit	1794e45e421fb5de03094f0b63f9ca95f86ab42f
Benchmark archive SHA-256	b5040097d2bfdd44600f3bf486fdb43eee3eb1247fec9c67900b5fcda6feb94a3
Derivative dataset SHA-256	1ba9575ff4a2b0342b2fa20afa242e052f263ff7bd3ed18734a923687a9c27d1
Qwen2.5-3B revision	aa8e72537993ba99e69dfaafa59ed015b17504d1
Qwen2.5-7B revision	a09a35458c702b33eeacc393d103063234e8bc28
Accuracy execution commit	456b864d6b0dd9dce5da5a5fdcb497bfc0510a37
Resource execution commit	6d0030094956626b17a9506d018e0fc077ca0ffd

E. Exploratory Resource Outcomes

The resource campaign uses 30 stratified held-out source-task clusters, all five variants, two models, four methods, and three repetitions, yielding 3,600 method-case measurements. All conditions were run on one NVIDIA GeForce RTX 3090 under frozen warm-up, thermal, process-isolation, and telemetry controls. For analysis, measurements are first averaged across the five variants within each source-task cluster. Repetitions remain separate.

Measured outcomes are complete method-case duration, input and output tokens, model calls, peak process RAM, peak board and process VRAM, and GPU-board energy. Utilization and temperature peaks are diagnostic. Resource contrasts use the same method pairs and 10,000-draw source-task-cluster bootstrap, but they are exploratory. GPU-board energy excludes the energy consumed by the rest of the workstation and does not estimate simulator, network, or UAV energy.

III. RESULTS

A. Descriptive Accuracy Outcomes

Figure 1 reports the two registered outcomes for all four methods. With Qwen2.5-3B, unsupported-continuation rates were 79.0% for M1, 0.4% for M2, 0% for M3, and 0.2% for M4. Nevertheless, all four 3B configurations had zero strict case success. The 3B M3 configuration was therefore conservative on the registered non-executable cases but did not satisfy the complete disposition and static-fidelity criteria.

With Qwen2.5-7B, unsupported-continuation rates were 68.3% for M1, 2.3% for M2, 0% for M3, and 2.6% for M4. Strict case-success rates were 0%, 0%, 16.1%, and 0.8%, respectively. In counts, M3 produced no unsupported continuation in 568 non-executable cases, whereas M1 produced 388 and M4 produced 15. M3 recorded 228 strict successes among all 1,420 cases, and M4 recorded 12.

The aggregate success rate must be read with its decomposition. All 228 M3 successes and all 12 M4 successes occurred on non-executable cases. Thus, M3 gave the fully correct disposition on 228/568, or 40.1%, of the non-executable cases, even though it contained all 568. The remaining contained cases did not meet every strict-success requirement. Across the 852 executable cases per method, every model-method combination had zero static plan fidelity. The experiment consequently demonstrates differences in containment and

TABLE IV
REGISTERED PAIRED ACCURACY CONTRASTS FOR QWEN2.5. VALUES ARE M3 MINUS THE LISTED COMPARATOR, WITH 95% SOURCE-TASK-CLUSTER BOOTSTRAP INTERVALS.

Model	Contrast	Estimate [95% interval]
<i>Unsupported continuation (lower is better)</i>		
Qwen2.5-3B	M3 — M1	−0.7905 [−0.8275, −0.7518]
Qwen2.5-3B	M3 — M4	−0.0018 [−0.0053, 0.0000]
Qwen2.5-7B	M3 — M1	−0.6831 [−0.7324, −0.6338]
Qwen2.5-7B	M3 — M4	−0.0264 [−0.0405, −0.0141]
<i>Strict case success (higher is better)</i>		
Qwen2.5-3B	M3 — M1	0.0000 [0.0000, 0.0000]
Qwen2.5-3B	M3 — M4	0.0000 [0.0000, 0.0000]
Qwen2.5-7B	M3 — M1	0.1606 [0.1507, 0.1697]
Qwen2.5-7B	M3 — M4	0.1521 [0.1415, 0.1627]

TABLE V
POST-HOC SESSION SUMMARY. EACH ENTRY IS A COUNT OUT OF 15 HELD-OUT SESSIONS. “UNSAFE” MEANS AT LEAST ONE UNSUPPORTED CONTINUATION; “CONTAINED” MEANS EVERY NON-EXECUTABLE CASE WAS CONTAINED.

Model	Method	Unsafe	Contained	Any strict
3B	M1	15	0	0
3B	M2	2	13	0
3B	M3	0	15	0
3B	M4	1	14	0
7B	M1	15	0	0
7B	M2	7	8	0
7B	M3	0	15	15
7B	M4	7	8	8

correct non-executable handling, not successful executable multi-UAV planning.

B. Registered Paired Contrasts

Table IV gives the registered paired differences. For Qwen2.5-7B, M3 reduced unsupported continuation relative to both M1 and M4, while increasing the registered strict case-success aggregate. For Qwen2.5-3B, the large containment difference relative to M1 did not translate into any strict success. The 3B M3 minus M4 unsupported-continuation interval reached zero.

A post-hoc session summary was consistent with the row-level direction. Table V reports the number of held-out sessions with at least one unsupported continuation, complete containment of all registered non-executable cases, and at least one strict success. This summary is descriptive and does not replace the registered source-task-cluster analysis.

Registered Accuracy Outcomes by Model and Validation Pipeline

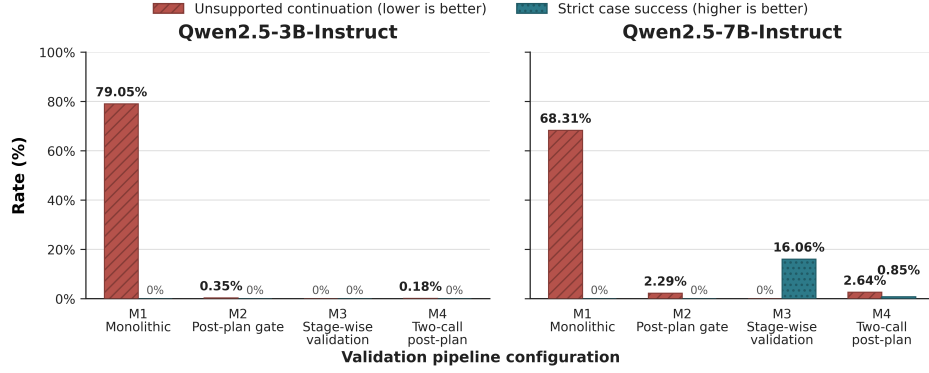


Fig. 1. Registered primary outcomes for the four configurations and two checkpoints. Unsupported continuation is evaluated on 568 non-executable cases per method; lower is better. Strict case success is evaluated on all 1,420 cases per method; every nonzero success bar consists exclusively of correct outcomes on non-executable cases.

C. Failure and Containment-Stage Analysis

Table VI decomposes the principal failure diagnostics. These diagnostics overlap and must not be summed. For example, a parse failure can also result in false non-execution. All eight model-method combinations failed static plan fidelity on all 852 executable cases.

The stage attribution in Table VII clarifies where each configuration terminated. “Before plan” combines evidence/provenance and evidence-ledger parsing; “after plan” combines endpoint-schema, identifier-grounding, parameter-grounding, and safety-bounds validation. Final-output parsing and model non-execution remain separate. The categories partition each model-method condition. For M3, the same zero-unsupported-continuation rate arose through markedly different termination behavior at the two model scales.

D. Exploratory Resource Contrasts

Relative to M1, 3B M3 added one model call, 3,297.68 input tokens, 260.69 output tokens, 8.86–9.49 s, and 1,565–1,681 J of GPU-board energy across the three repetitions. The bootstrap intervals for these differences were entirely positive. The 7B comparison had a different direction: M3 added one call and 3,155.56 input tokens but generated 125.72 fewer output tokens, required 6.23–6.82 s less time, and used 1,153–1,245 J less GPU-board energy. The duration, output-token, and energy intervals were entirely negative. Figure 2 retains each repetition and its source-task-cluster bootstrap interval.

Model-call difference was exactly zero for the M3 minus M4 comparison. For 3B, M3 used 179.62 more input tokens in every repetition, while duration, RAM, VRAM, output-token, and energy differences were small, inconsistent, or had intervals reaching zero outside the first repetition. For 7B, M3 used 72.17 fewer input tokens, 446.82 fewer output tokens, 31.41–35.99 s less time, approximately 41.5–59.9 MB less peak process RAM, and 5,669–6,279 J less GPU-board energy. The corresponding intervals were entirely negative. Board and process VRAM differences changed sign across repetitions and do not support a stable directional conclusion. The registered tables in the

public artifact snapshot retain all measured outcomes, including tokens, calls, and board and process VRAM.

IV. DISCUSSION

A. Containment Improved, but Executable Utility Did Not

The strongest result is a containment result. Both checkpoints show that the stage-wise configuration can prevent unsupported continuation on the registered non-executable cases. That behavior is important because it reduces the chance that a fluent but unsupported plan is passed downstream. It is not, however, sufficient for task utility. The 3B checkpoint demonstrates this directly: M3 contained every non-executable case but achieved no strict success. The 7B checkpoint gave 228 correct strict outcomes, but all occurred on non-executable cases. No tested method produced a statically valid plan on an executable case.

This decomposition changes the interpretation of the all-case success contrast. The positive 7B contrast does not mean that M3 completed more UAV missions. It means that M3 more often selected the fully correct non-executable disposition under the frozen static rubric. An executable planning system would need to preserve this containment behavior while also recovering schema, command, endpoint, and parameter fidelity on executable requests.

B. Configuration Effects and Model Scale

The comparison does not support a universal statement that additional validation must trade speed for containment. For 3B, M3 increased time and GPU-board energy relative to M1. For 7B, it reduced both despite an additional call. The 7B configuration also generated fewer output tokens than M1 and substantially fewer than M4. Shorter generation provides a plausible measured explanation for the lower latency and energy, but the experiment does not isolate that mechanism causally.

The M3 versus M4 comparison further shows that model-call parity is not resource parity. Prompt length, generated-token count, validation behavior, and termination patterns remain consequential. At the same time, these differences limit the

TABLE VI

POST-HOC FAILURE DIAGNOSTICS OVER 1,420 CASES PER MODEL AND METHOD. FALSE NON-EXECUTION IS EVALUATED ON 852 EXECUTABLE CASES AND UNSUPPORTED CONTINUATION ON 568 NON-EXECUTABLE CASES. COUNTS ARE DESCRIPTIVE AND MAY OVERLAP.

Model	Method	Parse errors	False non-execution	Unsupported continuation	Static-fidelity successes
3B	M1	368	249	449	0
3B	M2	368	851	2	0
3B	M3	544	852	0	0
3B	M4	369	848	1	0
7B	M1	429	267	388	0
7B	M2	429	850	13	0
7B	M3	273	852	0	0
7B	M4	392	850	15	0

TABLE VII

POST-HOC CONTAINMENT-STAGE ATTRIBUTION OVER 1,420 CASES PER CONDITION. COUNTS PARTITION EACH CONDITION AND ARE DESCRIPTIVE.

Model	Method	Before plan	After plan	Final parse	Model non-execution	Not contained	Accepted
3B	M1	0	0	368	0	449	603
3B	M2	0	1,049	368	0	2	1
3B	M3	971	0	445	1	0	3
3B	M4	0	1,046	369	0	1	4
7B	M1	0	0	429	18	388	585
7B	M2	0	958	429	18	13	2
7B	M3	37	0	267	1,115	0	1
7B	M4	0	920	392	91	15	2

causal interpretation of “placement”: M3 and M4 are complete pipeline alternatives, not the same validator moved to a different stage. A future isolation study should hold validation rules and prompt content fixed while changing only the point at which the rules are applied.

C. Implications for Multi-UAV Language Interfaces

The results support a layered design principle for language interfaces: a planning response should not be admitted merely because it is fluent or schema-shaped. Evidence availability, resource consistency, and provenance should be checked before a plan is offered to downstream software. Yet the zero executable-fidelity result also indicates that a containment layer cannot substitute for a capable task planner. Static admission control and executable planning should therefore be evaluated as separate system properties.

Before deployment claims are made, three additional forms of evidence are needed: manual auditing of executable outputs to quantify validator agreement, evaluation in an official server or simulator, and closed-loop testing under realistic sensing and communication uncertainty. Physical-UAV evaluation would additionally require independent motion, geofence, collision, and operator-authorization safeguards.

V. LIMITATIONS AND RESEARCH INTEGRITY

First, the five cases are controlled derivatives of MultiUAV-Plat source tasks, not live operator missions. Intervention wording or benchmark structure may produce cues that differ from naturally occurring failures. Second, the study uses two sizes from one model family; the observed scale dependence may not generalize to other checkpoints or decoding regimes. Third, static validators do not establish official-server, simulator,

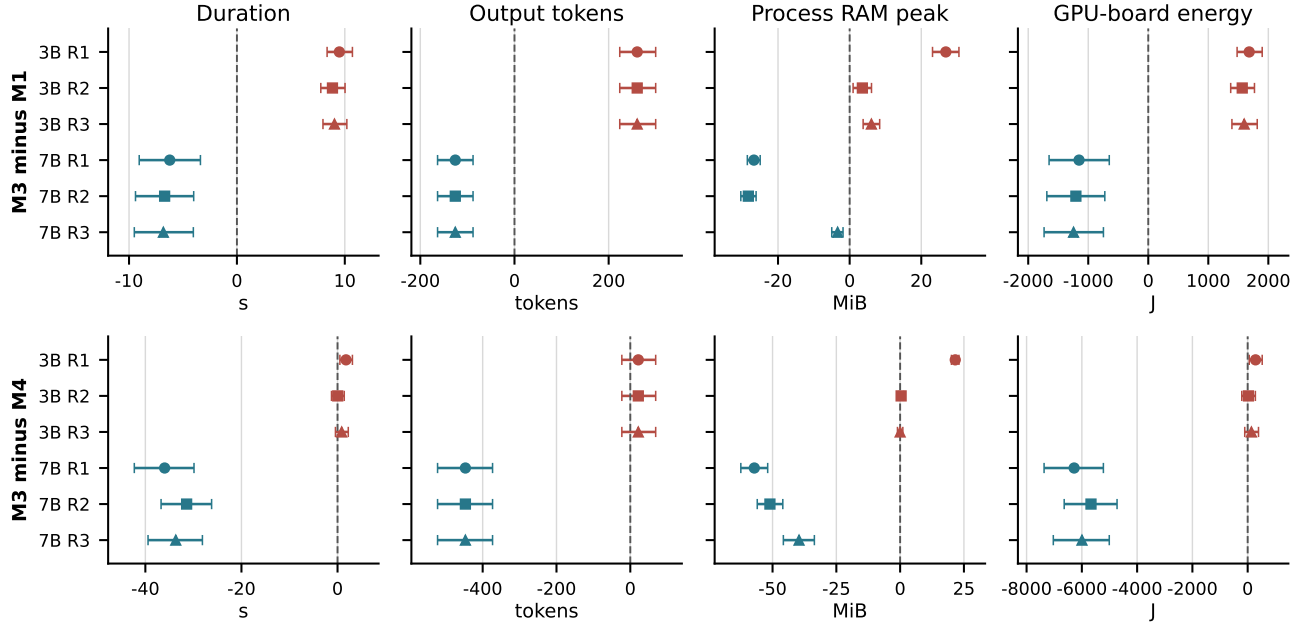
or physical-flight execution. The complete failure of executable fidelity could reflect model planning errors, validator mismatch, or both; the 30-cluster construction pilot is not a substitute for a stratified human audit of executable outputs.

Fourth, the experimental configurations differ in more than validation placement. In particular, M4 matches M3 only on model-call count. Fifth, accuracy uncertainty is clustered by source task even though source tasks are nested within 15 held-out sessions; the post-hoc session summary does not replace a session-clustered or hierarchical sensitivity analysis. Sixth, the resource study contains 30 source-task clusters, three repetitions, and one RTX 3090. Peak memory is an observed maximum rather than baseline-subtracted usage, and GPU-board energy is not whole-system energy.

Finally, a post-admission diagnostic displayed one row’s request and raw output before the resource aggregates were inspected. Execution and admission were already immutable, and the display exposed neither a hidden label nor an aggregate. This deviation is disclosed because the study does not claim that no human inspection of any admitted output occurred.

VI. CONCLUSION

Under a frozen static benchmark, the compared validation-pipeline configurations differed substantially in failure containment and local compute behavior. Stage-wise evidence and provenance validation eliminated unsupported continuation on the registered non-executable cases for both tested Qwen2.5 checkpoints. With Qwen2.5-7B it also increased the registered strict case-success aggregate relative to monolithic and model-call-count-matched configurations, but every success was a correct non-executable outcome. All methods had zero static plan fidelity on executable cases.



Point estimates and 95% source-task-cluster bootstrap intervals; repetitions are not pooled.

Fig. 2. Exploratory resource contrasts for M3 minus M1 (top) and M3 minus M4 (bottom). Points and error bars are repetition-specific paired cluster-mean estimates and 95% fixed-seed source-task-cluster bootstrap intervals over 30 clusters. Repetitions are not pooled. Positive values indicate that M3 used more of the named resource; negative values indicate less.

The defensible contribution is therefore a paired containment-and-measurement protocol and its mixed empirical result. The study does not show that stage-wise validation is universally cheaper, that placement alone caused the observed differences, or that the tested system can safely execute autonomous multi-UAV missions. Future work should isolate placement from other pipeline changes, audit validator agreement, and evaluate executable planning in an official environment before progressing to physical systems.

GENERATIVE AI USE DECLARATION

Generative AI tools were used to assist with manuscript organization, language refinement, $\text{\LaTeX}$ formatting, and figure presentation. The authors reviewed and verified all scientific content and retain full responsibility for the data, results, citations, interpretations, and conclusions. No generative AI system was treated as an author.

REFERENCES

- [1] A. Nazzari, R. Rubinacci, and M. Lovera, “TACOS: Task Agnostic Coordinator of a Multi-Drone System,” *Drones*, vol. 10, no. 4, Art. no. 251, 2026, doi: 10.3390/drones10040251.
- [2] A. Jarabo-Peñas, J. Bravo-Arrabal, E. G. A. Rolland, and A. L. Christensen, “Swarm-Steward: Scalable and Reliable Natural-Language Coordination of Autonomous Aerial and Ground Robots,” in *Proc. Int. Conf. Unmanned Aircraft Systems (ICUAS)*, 2026, 9 pp.
- [3] M. Zhao, S. Jain, and S. Song, “RoCo: Dialectic Multi-Robot Collaboration with Large Language Models,” arXiv:2307.04738, 2023.
- [4] A. Jiao *et al.*, “Swarm-GPT: Combining Large Language Models with Safe Motion Planning for Robot Choreography Design,” arXiv:2312.01059, 2023.

- [5] S. Nayak *et al.*, “LLaMAR: Long-Horizon Planning for Multi-Agent Robots in Partially Observable Environments,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024.
- [6] Y. Xie, C. Yu, T. Zhu, J. Bai, Z. Gong, and H. Soh, “Translating Natural Language to Planning Goals with Large-Language Models,” arXiv:2302.05128, 2023.
- [7] M. Majid and A. Y. Majid, “CommandSwarm: Safety-Aware Natural Language-to-Behavior-Tree Generation for Robotic Swarms,” arXiv:2605.07764, 2026.
- [8] S. Zhang, Q. Li, Y. Zang, X. Huang, Y. Fu, and C. Zhu, “MultiUAV-Plat: An LLM-Oriented Platform, Benchmark and Framework for Multi-UAV Collaborative Task Planning,” arXiv:2606.31073, 2026.